DATA STRUCTURES AND ALGORITHMS

Case Study

Unit II - Linear Data Structures, Searching and Sorting

Social Network Adjacency Matrix

Name	Rudrakshi Koul
Roll No.	09
College	Bharati Vidyapeeth's College of Engineering, Lavale

Case Study: Social Network Adjacency Matrix

Concept: 2D Arrays and Matrix Representation

Problem Statement

Represent friendship connections among users of a social network and answer queries such as "Are A and B friends?" and "How many friends does A have?"

Approach

- Store users as indices 0 to n-1.
- Use an n x n 2D array M where M[i][j] = 1 if users i and j are friends and 0 otherwise.
- For undirected friendship, the matrix is symmetric.

	Asha	Ravi	Meera	Karan
Asha	0	1	1	0
Ravi	1	0	0	1
Meera	1	0	0	1
Karan	0	1	1	0

Operations and Complexity

Operation	Time
Check friendship	O(1)
Count friends of a user (row sum)	O(n)
Add / remove friendship	O(1)
Space	$O(n^2)$

Limitation and Improvement

Real social networks are sparse, since most users are not friends with each other, so most cells are 0. A **sparse matrix representation** (storing only non-zero entries) or an adjacency list saves a large amount of memory.

Conclusion: A 2D array gives very fast lookups, but its $O(n^2)$ space cost makes sparse representations better for large networks.

Python Program

```
# Unit II: Social network using an adjacency matrix
users = ["Asha", "Ravi", "Meera", "Karan"]
n = len(users)
M = [[0] * n for _ in range(n)]

def add_friend(a, b):
    i, j = users.index(a), users.index(b)
    M[i][j] = M[j][i] = 1          # undirected: matrix stays symmetric
```

```
def are_friends(a, b):
    return M[users.index(a)][users.index(b)] == 1

def friend_count(a):
    return sum(M[users.index(a)])

for a, b in [("Asha", "Ravi"), ("Asha", "Meera"),
             ("Ravi", "Karan"), ("Meera", "Karan")]:
    add_friend(a, b)

print("      " + "  ".join(f"{u:5}" for u in users))
for u, row in zip(users, M):
    print(f"{u:6}" + "  ".join(f"{v:5}" for v in row))
print("Asha & Ravi friends?  ", are_friends("Asha", "Ravi"))
print("Asha & Karan friends? ", are_friends("Asha", "Karan"))
print("Friends of Karan:", friend_count("Karan"))
```

Output

```
      Asha   Ravi   Meera  Karan
Asha      0      1      1      0
Ravi      1      0      0      1
Meera     1      0      0      1
Karan     0      1      1      0
Asha & Ravi friends?   True
Asha & Karan friends?  False
Friends of Karan: 2
```